

题目链接

本题是2007年ICPC欧洲区域赛西北赛区的G题

题意

给定一个 $h \times w$ 的地图 ($1 \leq w, h \leq 500$)，每个位置有一个高度值，现在要求出这个图上的峰顶有多少个。峰顶是这样定义的：对于给定 d 值，一个高度为 h 的位置，如果它不经过不大于高度为 $h-d$ 的位置就无法走到更高的山峰，那么它就是峰顶。

分析

如果数据规模小，对每一个点做dfs即可求解，但是题目交代的数据规模大，需要做优化，优化的思路是贪心加bfs：先将所有点按照高度递减排序，再依次遍历每个点做bfs。

具体来说，用数组 $f[w \times h]$ 记录每个点在给定 d 值条件下可以到达的最大高度（初始值全为负，代表未求解且未遍历到）。然后按高度递减排序后依次做dfs遍历，开始时计数 $cc=1$ ，能到达的最大高度 $gx = g$ ，初始点入队列，初始点（记其高度为 g ）在 d 值限定下可以经过的点 v 有以下情况：1、 $f[v] > gx$ ，说明初始点不是峰顶， $gx = f[v]$ ；2、 $f[v] < 0$ ，则加入队列且赋值 $f[v] = g$ ，并且若 $h[v] == g$ ，则 $++cc$ 。bfs结束时：若 $gx == g$ ，说明本次找到 cc 个峰顶；否则本次未找到峰顶。

这里贪心的点在于：所有遍历到的点都是在最大下降高度 $< d$ 的前提下进行的，将高度降序排列了，后遍历点经过已遍历点时所有遍历过的点必然满足高度限制，所以后遍历到的点走到已遍历的非峰顶的点时能判定出此轮高度不超过初始点的都不是峰顶。

分析一下时间复杂度：记 $n=w \times h$ ，其实所有bfs做完仍然是 $O(n)$ 时间复杂度，排序导致整体时间复杂度为 $O(n \log n)$ 。

AC代码

```
#include <iostream>
#include <algorithm>
using namespace std;

#define N 252004
int h[N], s[N], f[N], q[N], t, r, c, d;

bool cmp(int i, int j) {
    return h[i] > h[j];
}

int bfs(int u, int g) {
    int head = 0, tail = 1, gd = g-d, cc = 1, gx = g; f[q[0] = u] = g;
    while (head < tail) {
        u = q[head++];
        int x = u/c, y = u%c, v;
        if (x > 0 && h[v = (x-1)*c + y] > gd)
            if (f[v] < 0) {
                if (h[v] == g) ++cc;
                q[tail++] = v; f[v] = g;
            } else gx = max(g, f[v]);
        if (x+1 < r && h[v = (x+1)*c + y] > gd)
            if (f[v] < 0) {
                if (h[v] == g) ++cc;
                q[tail++] = v; f[v] = g;
            } else gx = max(g, f[v]);
        if (y > 0 && h[v = x*c + y-1] > gd)
            if (f[v] < 0) {
                if (h[v] == g) ++cc;
                q[tail++] = v; f[v] = g;
            } else gx = max(g, f[v]);
        if (y+1 < c && h[v = x*c + y+1] > gd)
            if (f[v] < 0) {
                if (h[v] == g) ++cc;
                q[tail++] = v; f[v] = g;
            } else gx = max(g, f[v]);
    }
    if (gx > g) while (tail--) f[q[tail]] = gx;
    return gx > g ? 0 : cc;
}

void solve() {
    cin >> r >> c >> d;
    for (int i=t=0; i<r; ++i) for (int j=0; j<c; ++j) f[s[t] = t] = -1, cin >> h[t++];
    sort(s, s+t, cmp);
    int ans = 0;
    for (int i=0; i<t; ++i) if (f[s[i]] < 0) ans += bfs(s[i], h[s[i]]);
    cout << ans << endl;
}

int main() {
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    int t; cin >> t;
    while (t--) solve();
    return 0;
}
```